Newton School

High-Throughput Inference

Serving LLMs at Scale

What managed APIs hide. What production systems require.
What Chronicle's 5-agent swarm demands from day one.

KV Cache | PagedAttention | Continuous Batching | TTFT · TPOT

Newton School

WHAT WE COVER TODAY

From API Caller to Infrastructure Architect

PART 1
The Naive Serving Trap
Why every first attempt fails under concurrent load

PART 2
The KV Cache
Why memory is the real monster, not compute

PART 3
PagedAttention
The OS insight that changed everything

PART 4
Continuous Batching
The iteration-level revolution

PART 5
TTFT & TPOT
The two metrics defining whether users trust your system

CHRONICLE BUILD
5-Agent Swarm Logic
Calculating what the swarm actually needs

THE ILLUSION

It Works on My Machine.

THE PROTOTYPE PHASE

Engineer has working prototype. Gemini API responds in 800ms. Code is clean. Demo is perfect. Team is confident.

THE COST-SAVING PIVOT

They self-host the same model for cost reasons. First deployment: 3 concurrent users.

THE "VRAM" WALL

"CUDA out of memory. Killed."

They buy a bigger GPU. 5 concurrent users.

"CUDA out of memory. Killed."

THE REALITY CHECK

They are not hitting a hardware limit.

They are hitting an architecture limit they never knew existed.

This session closes the gap between calling an API and understanding what that API is doing for you.

The Two Ways Naive Serving Fails

Both feel like hardware problems. Neither is.

FAILURE MODE 1

Sequential Queue Collapse

Process one request at a time. Each takes 5s avg.

```
User 1: waits 0s ✓  |  User 2: waits 5s
User 3: waits 10s   |  User 4: waits 15s
User 10: waits 45s ✗
```

Enterprise SLA: sub-3 second response.
Violates SLA for 90% of users immediately.

Server functions correctly, but operationally fails everyone.

FAILURE MODE 2

Concurrent VRAM Exhaustion

Launch all requests concurrently. Each gets its own KV cache.

```
7B weights: 14 GB  |  Req 1-3 KV cache: 6 GB (2GB ea)
Total at Request 3: 20 GB

A10G GPU VRAM: 24 GB. Available: 10 GB.
→ Crash on Request 4.
```

Error message: "CUDA out of memory"
Actual cause: allocation strategy failure.

The solution to both failures is not more hardware. It is a different memory management architecture.

Newton School

The Thundering Herd Problem

Why traffic spikes kill naive servers faster than sustained load.

What triggers a Thundering Herd

- Scheduled batch jobs firing at the same clock time
- Cache expiration causing every client to retry at once
- Marketing campaign driving sudden traffic spikes
- Server restart - all queued clients retry simultaneously

The retry death spiral

Abandoned requests retry. Retry traffic adds to incoming rate. Queue grows faster than the initial herd alone would cause.

"The server is not being attacked. It is being killed by its own users trying to use it normally."

Design and test for P99

Average load never crashes servers. Peak load does.

The async harness we build today simulates this exact scenario at configurable concurrency levels.

01 **Server throughput: 0.2 req/s**
Processes one request every 5 seconds

02 **Herd arrives at 10 req/s**
Queue grows by 9.8 requests every second

03 **After 60 seconds**
Queue depth: 588 pending requests

04 **Last user waits 49 minutes**
They abandon. They retry. They make it worse.

Newton School

LLM Inference Is Not a Compute Problem

Every GPU you buy for more FLOPS is a GPU wasted on the wrong bottleneck.

The Roofline Model Applied to LLMs

A100 SXM: 312 TFLOPS compute | 2 TB/s HBM bandwidth

To utilize tensor cores, every byte must be used in ≥156 FP operations. Transformer arithmetic intensity is only ~1–10 ops/byte.

→ LLM inference is memory-bandwidth-bound.
The GPU is starved for data, not compute.

THE ANALOGY

F1 Car Behind a Truck

1,000 HP engine (FLOPS). Single-lane road (Bandwidth). Truck blocking (KV Cache).

- Road width = Memory bandwidth
- Truck size = KV cache footprint
- Adding HP = Buying more FLOPS (Irrelevant)

Optimization Outcomes

✘ **Faster GPU (more FLOPS)**

No measurable gain for inference workloads.

✔ **More Memory Bandwidth**

Direct throughput gain. This is why A100 bandwidth matters most.

✔ **Reducing KV Cache Footprint**

Directly enables higher concurrency and throughput.

✔ **Eliminating Fragmentation**

Ensures every byte of HBM is utilized effectively.

The KV Cache - Why It Exists

Without it: quadratic compute cost. With it: a new memory management problem.

KV CACHE — WHAT GETS STORED PER TOKEN

```
# For a 7B model (Llama-3-8B) with 32 layers, 32 heads, dim 128
kv_per_token = (
    2           # K matrix + V matrix
  * 32          # attention layers
  * 32          # attention heads
  * 128         # head dimension
  * 2           # bytes (float16)
) # = 524,288 bytes = 512 KB per token

# Real conversation footprint:
tokens_2k   = 2_000 * 512_000   # = 1.0 GB
tokens_32k  = 32_000 * 512_000  # = 16.0 GB
tokens_128k = 128_000 * 512_000 # = 64.0 GB

# Without KV cache: recompute ALL previous tokens each step (Quadratic)
# With KV cache: store once, read once per step (Linear) ✓
```

The KV cache eliminates quadratic recomputation but introduces an unpredictable, growing memory footprint. Response length is unknown at request start. This unpredictability is the root cause of fragmentation.

KV Cache Memory by Model Size

The numbers that determine how many agents Chronicle can run simultaneously.

MODEL	LAYERS	HEADS	KV PER TOKEN	CACHE @ 2K	CACHE @ 8K
Llama-3-8B	32	32	512 KB	1.0 GB	4.0 GB
Llama-2-13B	40	40	800 KB	1.6 GB	6.4 GB
CodeLlama-34B	48	64	1.5 MB	3.0 GB	12.0 GB
Llama-3-70B	80	64	2.5 MB	5.0 GB	20.0 GB
Mixtral-8×7B	32	32	512 KB	1.0 GB	4.0 GB

The unpredictability problem

System doesn't know response length at request start.

- 50 tokens (quick answer) → 25 MB KV cache
- 2,000 tokens (detailed analysis) → 1 GB KV cache

Pre-allocating for max wastes VRAM for short responses. Pre-allocating for min forces expensive reallocation. Both cause massive waste.

Chronicle's 5-agent VRAM budget

Chronicle runs 5 agents simultaneously. Each holds its own cache.

- `Model weights: 14 GB`
- `5× KV cache @ 4K: 20 GB (5 × 4 GB)`
- `Safety headroom: 8 GB`
- **`Minimum VRAM: 42 GB`**

Minimum: A100 40GB (tight) or A100 80GB (recommended). Session 11.2 covers how quantization changes this.

Memory Fragmentation - The Silent Throughput Killer

The server reports OOM. The VRAM monitor shows 50% free. Both are right.

VRAM ADDRESS SPACE — FRAGMENTATION SCENARIO

```
# 80 GB A100. After several requests complete:
# ▓▓▓
```

Newton School

PagedAttention - The Operating System Insight

The solution was in every OS textbook for 60 years. In 2023, someone applied it to transformers.

OS Virtual Memory

Physical RAM is divided into fixed-size **pages (4KB)**. The OS page table maps virtual pages to physical frames anywhere in RAM.

The process sees a contiguous virtual address space, remaining completely unaware of the underlying physical fragmentation.

The PagedAttention Logic

VRAM is divided into fixed-size **KV blocks** (16 tokens each). A block table maps logical token positions to physical VRAM blocks.

- Contiguous logical KV cache
- Non-contiguous physical VRAM allocation
- Zero external fragmentation

THE ANALOGY

Library Card Catalog

Old: Assign one contiguous shelf section per patron. Most stays empty.

Paged: Assign slots as books are acquired. Card catalog tracks locations.

Result: 4× more patrons; no wasted space.

Bonus: Prefix Caching

Shared prompts stored once. Reduces KV cache usage by 30–60%.

The 8× Throughput Multiplier

A fundamental cost architecture decision with measurable dollar value.

CRITERIA	NAIVE (CONTIGUOUS)	PAGEDATTENTION	IMPROVEMENT
Memory allocation	Contiguous, pre-alloc at max	Non-contiguous, on-demand	**4–8× more concurrent reqs**
VRAM utilization	12–20% effective	80–96% effective	**5–8× less waste**
Batch strategy	Sequential or static	Iteration-level	**23× throughput gain**
Concurrent requests	33 @ max 4K context (80GB)	264 @ avg 500 token resp	**8× capacity**
Infrastructure cost	8 GPUs for target throughput	1 GPU for same throughput	**8× cost reduction**

The Arithmetic (80GB A100)

Available for KV cache: 66 GB (after weights)

NAIVE (CONTIGUOUS)
33 requests @ 2GB each → 87.5% waste

PAGED (NON-CONTIGUOUS)
8,448 blocks → 264 concurrent requests

264 ÷ 33 = 8× improvement. Same hardware.

The Annual Dollar Value

Naive (8 A100s): $24/hr
Paged (1 A100): $3/hr

Annual savings: $183,960

Chronicle at Scale
5 agents share VRAM as a single block allocator, removing per-agent memory blocking.

Static Batching - 62% GPU Compute Wasted

The batch waits for the slowest request. The GPU computes padding. Nobody wins.

❌ STATIC BATCHING

```
# Static batch of 4 requests (A: 5, B: 50, C: 200, D: 500 tokens)
# The GPU processes ALL 4 until D finishes.

# Step 6:   A is done. Slot occupied by padding.
# Step 51:  B is done. Slot occupied by padding.
# Step 201: C is done. Slot occupied by padding.
# Step 500: D finishes. Batch drains.

# Wasted steps: A(495) + B(450) + C(300) = 1,245 / 1,997 (62%)
# GPU has apparent capacity, but new users wait the entire time.
```

✅ WHAT SHOULD HAPPEN

```
# Iteration-level scheduling:
# Step 1:   A, B, C, D all active.
# Step 5:   A completes. Slot freed immediately.
# Step 6:   E enters the freed slot. Active: E, B, C, D.
# Step 50:  B completes. F enters. Active: E, F, C, D.
# Step 200: C completes. G enters.
# Step 500: D completes. H enters.

# GPU idle time: near zero. No padding. No barriers.
# New users enter the moment any slot frees.
# Empirical result: 23× higher throughput on realistic workloads.
```

Static batching maps to GPU SIMD architecture naturally. Continuous batching requires a custom iteration-level scheduler. The complexity is the engineering cost of the 23× gain.

Continuous Batching - Iteration-Level Scheduling

The GPU processes one step at a time. Every completed step is an opportunity to admit a new request.

01 Each iteration: one forward pass

Scheduler checks completed requests and admits waiting requests every single generation step.

02 Immediate slot reuse

A 5-token request finishes at step 5. A waiting request enters at step 6. No pause. No barrier synchronization.

03 Priority preemption via KV block swapping

Low-priority long request preempted by swapping KV blocks to CPU RAM. High-priority request takes slot. Preempted request resumes when GPU frees.

04 23× throughput - when it's largest

Gain is largest when output length variance is high. Chronicle's agents have very high output variance.

Comparison table

CRITERIA	STATIC	CONTINUOUS
GPU idle time	High	~Zero
Queue wait	Full batch	Immediate
Padding waste	62%	Zero
Throughput	1×	Up to 23×

Chronicle implication

Chronicle's 5 agents vary wildly in output length (Ingestion: 200, Brutality: 1,500).

Static batching would idle 4 agent slots waiting for Brutality to finish.

Continuous batching keeps all 5 slots productive on every iteration.

Newton School

TTFT & TPOT: The User Experience Metrics

Optimizing for one often sacrifices the other. Infrastructure design is the art of balance.

TTFT

Time To First Token

Wall-clock time from request dispatch to first token received. Sets initial perception of speed.

COMPONENTS

`Queue wait + Prefill pass = TTFT`

TARGET

`P50: < 500ms | P99: < 1,500ms`

OPTIMIZATION LEVER

Continuous batching reduces queue wait; shorter inputs reduce prefill cost.

TPOT

Time Per Output Token

Average wall-clock time per token after the first token. Determines streaming quality.

COMPONENT

GPU memory bandwidth per decode step.

TARGET

`< 30ms (P50) | < 50ms (P99)`

OPTIMIZATION LEVER

Larger batch size amortizes KV read cost, but increases TTFT.

The Opposing Pressure

- **Increasing batch size:** Better TPOT, worse TTFT.
- **Decreasing batch size:** Better TTFT, worse TPOT.

Optimal batch size is the point where TTFT SLA is just barely met while TPOT is minimized. Finding this requires load testing at expected peak concurrent user counts.

Where This Session Lives in Chronicle

Session 11.1 establishes the inference foundation every subsequent session builds on.

THIS SESSION (11.1)

Foundation

- VRAM budget calculated.
- KV cache mechanics understood.
- PagedAttention: why it exists.
- Continuous batching: why it matters.
- TTFT / TPOT: how to measure.
- Load testing harness: built.

NEXT (11.2)

Quantization

- Which Chronicle agents survive INT4?
- Which require FP16?
- VRAM budget recalculated per tier.
- Task survivability matrix applied.

AFTER THAT (11.3)

GPU Allocation

- max_model_len configured per agent.
- OOM prevention baked in.
- Tensor parallelism applied.
- Chronicle deployable without risk.

What the load test proves

The async harness produces real TTFT/TPOT measurements at peak concurrency. These are Chronicle's baseline. Session 11.2's quantization changes them. Session 11.3's GPU allocation locks them to a deployable configuration.

Newton School

SESSION 11.1 COMPLETE

You now understand what managed APIs hide from you.

CONCEPTS COVERED

- ✓ Why naive serving fails under concurrent load
- ✓ The Thundering Herd & queue depth mathematics
- ✓ Why LLM inference is memory-bound (roofline)
- ✓ KV Cache: mechanics & per-model sizing
- ✓ Memory fragmentation: 80GB vs 16GB efficiency
- ✓ PagedAttention: virtual memory for KV cache
- ✓ The 8× throughput multiplier on identical hardware
- ✓ Continuous batching: 23× gain via iteration-level scheduling
- ✓ TTFT and TPOT: defining user experience
- ✓ Chronicle's 5-agent VRAM budget: 42GB minimum

Next: In Session 11.2 →

We apply AWQ/GPTQ/GGUF quantization to Chronicle's agent swarm. The task survivability matrix decides which agents can run INT4 and which require FP16. The VRAM budget changes. The model changes. The architecture does not.